REST Trading-System Integration Specification

Status: proposed REST interface and integration specification - no implementation
Scope: a trading system calls the callable Bermudan cross-currency swap pricer over REST from its C++ API
Version: 1.0, 2026-08-14

1. Decision and boundary

Expose the pricer as a versioned REST service owned by this system. The trading system remains the system of record and uses its existing C++ API layer as the REST client. The service validates a complete, immutable pricing request, maps it into the existing QuantLib-oriented domain objects, runs calibration/pricing, and returns measures plus diagnostics. No QuantLib, Python, or Flask type crosses the REST boundary.

```
Trading system -> C++ REST client -> HTTPS/JSON -> pricing REST adapter
                                                -> validation and normalization
                                                -> QuantLib pricing engine
                                                -> versioned JSON result
```

The REST adapter must stay thin: authentication, schema validation, request routing, idempotency, serialization, and error translation only. Product construction, curves, calibration, Monte Carlo, LSM exercise, and risk calculations belong in the pricing layer. This preserves the repository convention that the web application does not own pricing logic.

The first deployment should be one WSGI service on the existing single host, behind Apache or another existing reverse proxy. Use the Flask development server only for local testing. Add distributed infrastructure only after measured demand requires it.

2. API surface

All routes are under `/api/v1`. JSON uses UTF-8, ISO `YYYY-MM-DD` dates, UTC ISO-8601 timestamps, decimal rates (`0.025`, not `2.5`), and FX quoted as domestic currency units per one foreign currency unit.

Method and route	Purpose	Success
`POST /api/v1/callable-xccy/validate`	Validate; return `ValidationResult`	`200`
`POST /api/v1/callable-xccy/price`	Bounded synchronous price; return `PriceResult`	`200`
`POST /api/v1/callable-xccy/jobs`	Submit long pricing/risk work; return `JobStatus` + `Location`	`202`
`GET /api/v1/callable-xccy/jobs/{job_id}`	Return one `JobStatus`, containing result/error when terminal	`200`
`DELETE /api/v1/callable-xccy/jobs/{job_id}`	Idempotently request cancellation; return `JobStatus`	`202/200`
`GET /api/v1/engine`	Return `EngineInfo`: versions, schemas, models, measures, limits	`200`
`GET /health/live`	Process liveness only	`200`
`GET /health/ready`	Dependency/config readiness	`200` or `503`

`ValidationResult` contains `valid`, normalized identifiers/conventions, warnings, and field errors. `JobStatus` contains `job_id`, status, timestamps, progress stage, `status_url`, and exactly one of `result` or `error` when terminal. Submission sets `Location` to `status_url`. The synchronous endpoint enforces the published server deadline and rejects configurations designated asynchronous. On deadline it returns `504`, requests cancellation, and never publishes a late value as an official result. Portfolio, full-risk, or high-path-count work uses jobs. Polling is the initial integration; callbacks and message queues are out of scope.

3. Transport and C++ client behavior

Use HTTPS with TLS 1.2 or later and mutual TLS for machine identity. The server maps the client-certificate identity to allowed environments, endpoints, and model configurations. Credentials live outside source control and never appear in logs. Network access is restricted to approved trading-system hosts.

The C++ client must:

- set `Content-Type: application/json` and `Accept: application/json`;
- send a unique `X-Request-ID` and an `Idempotency-Key` retained across retries; the body has no separate request ID;
- send `X-Correlation-ID` when the trading system has an end-to-end trace ID;
- set connection and total deadlines; never wait indefinitely;
- verify the server certificate and hostname;
- retry only the cases defined in Section 7, with exponential backoff and jitter; and
- parse error codes and fields, not free-text messages.

Compress large batch payloads with gzip only after an interoperability test. Place a configured limit on request bytes, nesting depth, trade count, and returned diagnostics.

4. Versioned request contract

The body contains four immutable blocks: `trade`, `market`, `model`, and `numerics`. Unknown schema versions and enum values are hard errors. Optional fields must have documented economic defaults; calendar, index, day count, exercise holder, settlement, and FX orientation may not be defaulted silently. The JSON below is abbreviated for readability; the versioned OpenAPI/JSON Schema produced during implementation is the machine-readable realization of these mandatory rules, not a license to change them.

```
{
  "schema_version": "callable-xccy-price/1.0",
  "valuation_date": "2026-08-14",
  "reporting_currency": "USD",
  "trade": {
    "trade_id": "XCCY-123", "trade_version": 7,
    "holder": "RECEIVE_DOMESTIC", "domestic_currency": "USD",
    "foreign_currency": "EUR", "fx_quote": "USD_PER_EUR",
    "domestic_leg": {}, "foreign_leg": {},
    "notional_exchanges": {}, "exercise_schedule": []
  },
  "market": {
    "snapshot_id": "MKT-20260814-CLOSE",
    "snapshot_time": "2026-08-14T21:00:00Z",
    "curves": {}, "swaption_vols": {}, "fx_spot": 1.10,
    "fx_vols": {}, "correlations": {}, "fixings": {}
  },
  "model": {"config_id": "HW1F_HW1F_BSFX_LSM_V1"},
  "numerics": {"config_id": "EOD_V1", "seed": 41719},
  "measures": ["NPV", "NON_CALLABLE_NPV", "OPTION_VALUE", "EXERCISE_PROFILE"]
}
```

The production schema must fully define both legs: signed notionals, pay/receive direction, schedules, calendars, business-day rules, day counts, indices, spreads/gearing, compounding, fixing/payment lags, stubs, initial/final exchanges, and historical fixings. Exercise events specify decision, effective, and settlement dates; cancellation/entry style; holder; fee; and same-day event ordering. Notionals and money use JSON numbers in the stated currency; rates/spreads are decimals; probabilities are in `[0,1]`; correlations are in `[-1,1]`; dates must be ordered and currency codes must be ISO 4217. Empty arrays/objects are rejected when the associated feature is required.

Holder cash flows are positive receipts and negative payments. `callable_npv` is from the named holder's perspective. `reporting_currency` must equal the domestic currency in version 1; arbitrary reporting-currency conversion is rejected rather than inferred. Conflicting schedule/exercise dates or same-day ordering are `INVALID_TRADE`.

The preferred market contract is an immutable snapshot by value or a snapshot ID whose content cannot change. It supplies domestic and foreign discount/forecast curves, collateral and cross-currency basis treatment, spot and quote direction, both swaption surfaces, FX Black volatilities, all three correlations, and fixings. Curve ownership is explicit: either the service bootstraps from raw instruments or receives curve nodes/discount factors. A request may not mix these modes silently.

Approved `model.config_id` and `numerics.config_id` values resolve to immutable server-side configurations. Any permitted override is echoed in the response and marked non-standard. The service computes `input_hash` over the entire canonical normalized semantic body, including trade, market, model, numerics, and requested measures; only authentication and transport metadata are excluded.

5. Result contract

A successful synchronous response, or a completed job, returns:

```
{
  "schema_version": "callable-xccy-result/1.0",
  "request_id": "8c4f...", "result_id": "PRC-...",
  "status": "SUCCEEDED", "input_hash": "sha256:...",
  "valuation_date": "2026-08-14", "currency": "USD",
  "measures": {
    "callable_npv": 125430.12, "non_callable_npv": 101100.33,
    "embedded_option_value": 24329.79,
    "mc_standard_error": 410.25, "confidence_level": 0.95
  },
  "exercise": {"by_date": [], "no_exercise_probability": 0.31},
  "diagnostics": {},
  "versions": {"engine": "...", "quantlib": "...", "model_config": "..."}
}
```

The echoed `request_id` is the `X-Request-ID`. Measure mapping is exact: request `NPV` returns `callable_npv`; `NON_CALLABLE_NPV` returns `non_callable_npv`; `OPTION_VALUE` returns `embedded_option_value`; and `EXERCISE_PROFILE` returns `exercise`. All monetary results are unrounded IEEE-754 JSON numbers in `currency`; display rounding belongs to the trading system. Required provenance includes market snapshot, normalized input hash, engine/build and QuantLib versions, model/numerical configuration, seed, training/pricing path counts, grid size, runtime, and calibration/regression status. Requested risks state unit, bump, method, recalibration choice, and whether the policy was retrained.

For jobs, `GET` returns `QUEUED`, `RUNNING`, `SUCCEEDED`, `FAILED`, or `CANCELLED`, plus timestamps and sanitized progress. Results remain retrievable for an agreed retention period. Cancellation is best effort and terminal jobs remain terminal.

6. Idempotency, replay, and concurrency

An idempotency key is scoped to client identity, environment, HTTP method, and route, and retained for 30 calendar days. The same key plus the same `input_hash` returns the original response: `202` with the same `Location` while running, the completed result after success, or the same terminal error/status after failure or cancellation. Reusing the key with any different semantic request body returns `409 IDEMPOTENCY_CONFLICT`. The server atomically claims a key before work begins.

Canonicalization sorts object keys, preserves array order, normalizes dates/currencies/enums, encodes finite numbers with a versioned round-trip representation, and excludes headers, authentication, and transport metadata. The response records the canonicalization version. `DELETE` is idempotent: repeated cancellation of a running job returns its current status; a terminal job is unchanged.

Persist job metadata using `job_id`, type, status, started/finished timestamps, error, and logs path. Store only what model governance and replay policy require. Do not put full proprietary payloads in ordinary logs.

QuantLib evaluation date and observable handles require isolation. A request creates one valuation context, sets the date once, and uses immutable market objects. Initial production concurrency should use a bounded process pool or serialized valuation contexts unless the exact QuantLib build has passed mixed-date thread-safety tests. Cache only immutable objects keyed by snapshot, configuration, engine, and QuantLib versions.

7. Errors and retry rules

Immediate request/transport failures use `application/problem+json` with `type`, `title`, `status`, stable `code`, `message`, `request_id`, `field_errors`, `retryable`, and optional sanitized diagnostics. `Retry-After` is seconds or an HTTP date. A job accepted with `202` is subsequently read with HTTP `200`; runtime calibration/numerical failure appears as `status=FAILED` and the same problem object in `JobStatus.error`. HTTP `4xx/5xx` on `GET` describes the GET operation itself, not a stored job failure.

HTTP	Stable code examples	C++ client action
`400`	`INVALID_JSON`, `INVALID_TRADE`, `INVALID_MARKET`	Correct request; no retry
`401/403`	`UNAUTHENTICATED`, `FORBIDDEN`	Refresh credentials once or escalate
`404`	`JOB_NOT_FOUND`	Verify environment/retention
`409`	`IDEMPOTENCY_CONFLICT`	Stop; investigate key reuse
`422`	`CALIBRATION_FAILED`, `NUMERICAL_FAILED`	No automatic config change; escalate
`429`	`CAPACITY_EXCEEDED`	Retry `GET`/submission per `Retry-After`
`503/504`	`UNAVAILABLE`, `DEADLINE_EXCEEDED`	Bounded retry with same idempotency key
`500`	`INTERNAL_ERROR`	One controlled retry, then escalate

Never convert a failed official calculation into a different model or lower-quality configuration without an explicit new request. QuantLib exceptions are caught at the pricing boundary and translated; stack traces are not returned to callers.

8. Security and operations

- authorize service identities by endpoint/environment and restrict non-standard model overrides;
- validate schemas before allocating large pricing objects;
- redact credentials and sensitive payload fields from structured logs;
- log request/job ID, trade ID, snapshot ID, input hash, stage, duration, versions, and stable error code;
- measure request/job counts, p50/p95/p99 latency, queue depth, failures, calibration error, regression rank/condition, Monte Carlo error, memory, and cache hits; and
- expose liveness independently of readiness so unhealthy workers leave rotation without restart loops.

Production must run a pinned, reproducible build under a non-privileged account. The reverse proxy terminates or passes through approved TLS, enforces body/time limits, and does not cache pricing responses. Backups and retention follow the trading platform's data classification.

9. Acceptance and rollout

Before writeback, demonstrate:

1. schema/enum round trips and exact unit, sign, calendar, index, and FX-orientation mappings from the C++ client;
2. validation, authentication, authorization, size limit, timeout, cancellation, retry, and idempotency behavior;
3. pricing parity with reviewed standalone fixtures within Monte Carlo confidence/tolerance, including non-callable, one-exercise, and full Bermudan cases;
4. deterministic replay on supported builds, plus correct engine/snapshot/config provenance;
5. concurrent mixed-date and stale/invalid-market isolation, soak behavior, and bounded resource use; and
6. rollback to the prior service/config while replaying the same stored request.

Roll out through local contract tests, a read-only integration environment, shadow pricing, controlled parallel comparison, then limited official use behind a feature flag. The trading system retains its current pricer as the operational fallback until model validation and platform owners sign off.

10. Explicit non-goals and open decisions

This specification does not implement endpoints, change source code, define proprietary C++ client classes, introduce callbacks/queues, or authorize result writeback. Before implementation, owners must approve the generated OpenAPI schema; certificate authority and rotation procedure; payload limits; synchronous deadline; job capacity; market-data ownership; callable payoff conventions; approved model/numerical configurations; service-level objectives; replay tolerance; and official-result workflow.

Companion quantitative definition: `docs/CALLABLE_BERMUDAN_XCCY_SWAP_PRICING_SPEC.md`.

Rollback

This is documentation only. Removing this Markdown file and generated PDF fully rolls back the change; no runtime behavior is affected.